Type-Directed Discretization of Probabilistic Programs

KATHERINE WU, Cornell University, USA
JULES JACOBS, Jane Street, USA
KEVIN BATZ, Cornell University, USA
ALEXANDRA SILVA, Cornell University, USA

We study *exact discretization* as the intended specification of a transformation for recursive, higher-order probabilistic programs with continuous distributions. We target programs where continuous values are compared against finitely many constants, so the transformation reduces inference to a discrete problem. Our central technical contribution is a non-local, type-directed analysis that infers where continuous values can be partitioned into finitely many observationally relevant regions, then rewrites sampling and comparison behavior over those regions. We call this transformation Slice. Because this construction is global and type-directed, correctness requires reasoning beyond the local syntax: we formalize the transformation and its operational semantics, and state the measurable weak compiler simulation and terminal-limit compatibility needed for soundness of boolean queries. These proof obligations remain open. Transformed programs can be executed by discrete engines such as Dice, Roulette, and Storm. Our empirical evaluation shows that Slice supports case studies combining continuous sampling, higher-order functions, and unbounded recursion and, on benchmarks where direct comparison is possible, is competitive with exact inference systems for continuous programs.

CCS Concepts: • **TODO**;

Additional Key Words and Phrases: probabilistic programming, exact inference, continuous distributions, discretization, type systems

ACM Reference Format:
Katherine Wu, Jules Jacobs, Kevin Batz, and Alexandra Silva. 2026. Type-Directed Discretization of Probabilistic Programs. 1, 1 (August 2026), 112 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 Introduction

Probabilistic programming languages (PPLs) are widely used to model uncertainty [6, 10, 14, 15, 17, 21, 24, 25, 27, 34, 35, 37, 39–41, 45–47]. Programs with *continuous* distributions capture rich models but require reasoning about uncountable domains, whereas purely *discrete* programs often admit fast and precise inference but cannot express real-valued models.

Slice studies *exact discretization*: a source-to-source transformation intended to preserve semantics while converting, if possible, continuous programs into discrete ones. We target higher-order recursive programs with conditioning and continuous sampling, without arithmetic. The key mechanism is *type-directed cut inference*: our type system tracks each constant threshold (or *cut*) used in comparisons and uses this information to partition continuous domains into finitely many intervals. For example, $\textbf{uniform}(0,1) < 0.5$ discretizes to $\textbf{discrete}(0.5, 0.5) < 1$, yielding a discrete program that existing exact inference tools can process.

Cuts may arise far from their source expressions: a sampled value may flow through higher-order functions, conditionals, recursive calls, data structures, and observations before it is eventually compared. To discretize soundly, Slice propagates cut information along these non-local flows, collecting all observationally relevant cut points for each continuous value and using them to partition

Authors' Contact Information: Katherine Wu, kaw324@cornell.edu, Cornell University, Ithaca, New York, USA; Jules Jacobs, julesjacobs@gmail.com, Jane Street, New York, New York, USA; Kevin Batz, ksb239@cornell.edu, Cornell University, Ithaca, New York, USA; Alexandra Silva, alexandra.silva@cornell.edu, Cornell University, Ithaca, New York, USA.

2026. ACM XXXX-XXXX/2026/8-ART
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

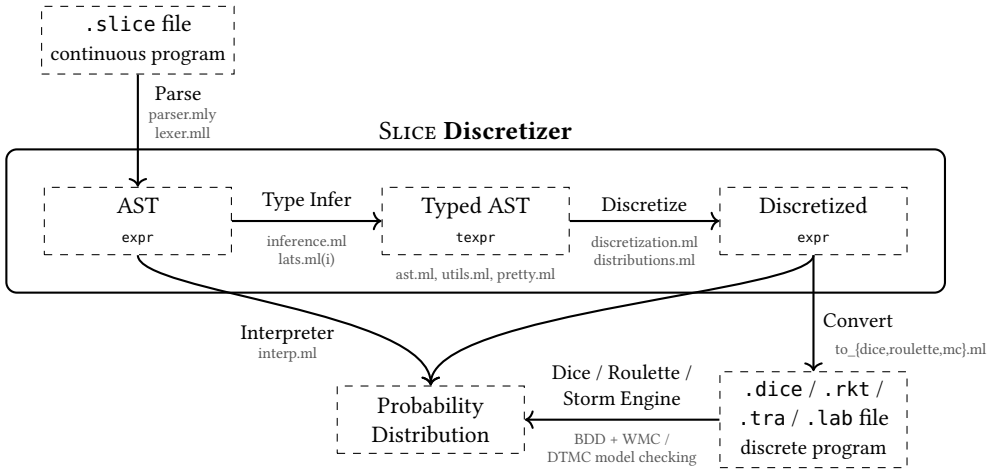


Fig. 1. Pipeline for exact discretization: typed Slice programs are transformed into semantically equivalent discrete programs, then evaluated by discrete engines. The transformation stage (type inference + discretization) is backend-agnostic; Dice, Roulette, and Storm are examples of target backends.

its domain into finitely many intervals (see §2 for illustrative examples). A type system provides a natural framework for this analysis: it gives a compositional account of how cut information propagates through each language construct (via one typing rule per langauge feature); it separates the analysis result, i.e. a type derivation, from the type inference algorithm used to infer it; and it supports our correctness argument, since semantic preservation can be stated as a logical relation indexed by our types carrying cut and value sets.

Our discretization method is subject to a principled restriction: a continuous value must ultimately be compared against constants; direct comparisons between two unknown reals such as $\texttt{uniform}(0,1) < \texttt{uniform}(0,1)$ cannot be discretized exactly and remain continuous. Nonetheless, benchmarks from the literature and our novel case studies show that many programs *can* be discretized by our system automatically.

Discretization itself is not new. Garg et al. [20] encode real-valued variables via user-chosen bit-precisions, trading accuracy for tractability. Other approximate discretization schemes exist in the literature [5, 12, 28]. In Slice, *exact discretization* names the specification that the discrete program preserves the relevant probabilities of the original continuous program. This differs from approximate discretization, which instead targets error bounds or convergence guarantees . The weak compiler simulation and terminal-limit compatibility needed to establish this specification remain open proof obligations.

Figure 1 shows the concrete pipeline from typed source programs to discrete backends. Because discretization is source-to-source, any discrete PPL that supports finite distributions can serve as a backend; we instantiate this with Dice [27], Roulette [35], and Storm [16].

Outline. Starting from examples (§2) and concluding with related and future work in §8, we:

Type System (§3) Characterize when programs are discretizable.

Type Inference (§4) Infer types automatically to guide discretization.

Exact Discretization (§5) Define a type-directed transformation from continuous programs to discrete ones using inferred type information.

Soundness (§6) Define the operational semantics and state the measurable weak compiler simulation and terminal-limit compatibility required to prove semantic preservation [7, 13, 48].

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}} \qquad \textsc{Discrete}\ \frac{}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)} \qquad \textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)}$$

$$\textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \mathbf{bool}}$$

$$\textsc{FinEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 ==_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 7. Typing rules for general language constructs

intervals know on which side of c they fall. For example, $2.3 < e$ requires cuts separating $(-\infty, 2.3]$ from $(2.3, \infty)$; finer partitions are allowed but not required.

In general, the typing rule for $e_1 < e_2$ requires that the expressions share a cut set B, and that their respective value sets V_1, V_2 are related in a way that allows the comparison to be determined after discretization, via the $\text{answers}_<(B, V_1, V_2)$ relation:

Definition 3.3. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_<(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {<}x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {\le}x \in B$.

To understand this definition, we first want to introduce two different perspectives on the B set. The first perspective is that B specifies a set of intervals, such that values of the expression will be mapped to their respective interval index. The second perspective is that this discretized index *still contains enough information to evaluate all comparisons in B when viewed as predicates*. That is, if B contains ${<}x$ then the discretized value of $e : \mathbf{float}[B; V]$ still contains enough information to answer $e < x$ precisely: if after discretization, e has an interval index below the interval containing x, then the comparison is true, and otherwise it is false.

Now, if $e_1 : \mathbf{float}[B; V_1]$ and $e_2 : \mathbf{float}[B; V_2]$, then $\text{answers}_<(B, V_1, V_2)$ ensures that the comparison result can still be determined after discretization because:

(1) If $B = \top$, then no discretization is performed, and the comparison can just be performed on the real-valued expression.
(2) If $B \neq \top$ and $V_2 \neq \top$, then the expression is discretized into intervals as determined by B, but in this case the right-hand side can only take on values in V_2. In this case, all ${<}x$ for $x \in V_2$ must be contained in B, which means that the value of e_2 can be exactly recovered from its interval index. Then, given the previous observation, we can determine the comparison result by comparing the interval index of e_1 to the interval index of e_2.
(3) If $B \neq \top$ and $V_1 \neq \top$, then the situation is similar to the previous case.

Note that $B = \top$ iff $V = \top$ accounts for comparisons where a continuously ranging expression is compared against arbitrary real values not statically known, such as other continuous distributions, in which case no discretization can be performed. Additionally, we insist that $B_1 = B_2$, which ensures that both comparands will be discretized to the same type, or not discretized at all.

Finally, for non-strict comparisons, the rule is analogous to Definition 3.3:

Definition 3.4. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_\le(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {\le}x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {<}x \in B$.

$e : \tau$	Condition	$\mathcal{D}[\![e : \tau]\!]$
$x : \tau$		x
c : **float**$[B; V]$	$B = \top$	c
	$B \neq \top$	$k_{\#(\lvert B\rvert+1)}$ such that $c \in \text{Intervals}(B)_k$
let $x = e_1 : \tau_1$ **in** $e_2 : \tau_2 : \tau_2$		**let** $x = \mathcal{D}[\![e_1 : \tau_1]\!]$ **in** $\mathcal{D}[\![e_2 : \tau_2]\!]$
(**if** e_1 : **bool then** $e_2 : \tau$ **else** $e_3 : \tau$) : τ		**if** $\mathcal{D}[\![e_1 : \textbf{bool}]\!]$ **then** $\mathcal{D}[\![e_2 : \tau]\!]$ **else** $\mathcal{D}[\![e_3 : \tau]\!]$
$(e_1 : \textbf{float}[B; V_1] < e_2 : \textbf{float}[B; V_2])$: **bool**	$B \neq \top$	$\mathcal{D}[\![e_1 : \textbf{float}[B; V_1]]\!] <_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \textbf{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \textbf{float}[B; V_1]]\!] < \mathcal{D}[\![e_2 : \textbf{float}[B; V_2]]\!]$
$(e_1 : \textbf{float}[B; V_1] \leq e_2 : \textbf{float}[B; V_2])$: **bool**	$B \neq \top$	$\mathcal{D}[\![e_1 : \textbf{float}[B; V_1]]\!] \leq_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \textbf{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \textbf{float}[B; V_1]]\!] \leq \mathcal{D}[\![e_2 : \textbf{float}[B; V_2]]\!]$
uniform(e_1 : **float**$[B_1; V_1]$, e_2 : **float**$[B_2; V_2]$) : **float**$[B; V]$	$e_1, e_2 \in \mathbb{R},\ e_1 > e_2$	**diverge** : τ
	$B = \top$	**uniform**($\mathcal{D}[\![e_1 : \textbf{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \textbf{float}[B_2; V_2]]\!]$)
	$e_1, e_2 \in \mathbb{R}$, $B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$	**discrete**$(p_0, \ldots, p_n)$ where $p_k = \text{CDF}(b_{k+1}) - \text{CDF}(b_k)$
	$B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, $V_1 = \{v_1, \ldots, v_n\}$, $V_2 = \{w_1, \ldots, w_m\}$	**let** $v_1' = \mathcal{D}[\![e_1 : \textbf{float}[B_1; V_1]]\!]$ **in** **let** $v_2' = \mathcal{D}[\![e_2 : \textbf{float}[B_2; V_2]]\!]$ **in** **if** $v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_1 : \textbf{float}[B_1; V_1]]\!]$ && $v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_1 : \textbf{float}[B_2; V_2]]\!]$ **then** $\mathcal{D}[\![\textbf{uniform}(v_1, w_1) : \textbf{float}[B; V]]\!]$ ... **else if** $v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_i : \textbf{float}[B_1; V_1]]\!]$ && $v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_j : \textbf{float}[B_2; V_2]]\!]$ **then** $\mathcal{D}[\![\textbf{uniform}(v_i, w_j) : \textbf{float}[B; V]]\!]$ ... **else** $\mathcal{D}[\![\textbf{uniform}(v_n, w_m) : \textbf{float}[B; V]]\!]$

Fig. 13. Discretization function, where $\text{CDF}(b_0) = 0$ and $\text{CDF}(b_{n+1}) = 1$. For the **uniform** distribution, $V = \top$ is implicit, and all remaining cut-set and value-set cases are ruled out by the constraints enforced during type inference. For all other language constructs, $\mathcal{D}[\![\cdot]\!]$ recurses structurally over them.

Additional constraint for contextual equivalence. When the return type of the program involves a float type, either directly or as part of a larger compound type (e.g., **float**$[B; V] \to$ **float**$[B'; V']$), we add an additional constraint to ensure that these float types are not discretized. We recursively traverse the return type and set all nested B sets to $\top$. This ensures that the observable behavior of the program is preserved after discretization, regardless of the context in which the program is used. When the return type is basic, such as **bool**, nothing needs to be done.

Example 4.2. To make the constraint solving process concrete, Figure 12 walks through §2.3. Each constraint obtained from the generation phase is processed by the solver, then collected to obtain the final cut and value sets.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression e : **bool** and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \textbf{bool}]\!]$: **bool** such that e and $\mathcal{D}[\![e : \textbf{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this intended semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. For the uniform fragment defined in §6, Theorem 6.1 states this equality conditional on the remaining weak compiler-simulation

obligation. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and discretization makes that reduction explicit.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; §6 states the remaining proof obligation for this non-local, type-mediated construction . Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e = \mathbf{uniform}(e_1, e_2)$ of type $\mathbf{float}[B; V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression $e' : \mathbf{fin}(n+1)$ such that the probability that e evaluates to a value in $\mathrm{Intervals}(B)_i$ *coincides* with the probability that e' evaluates to $i_{\#n+1}$. In particular, if *all* $\mathbf{float}[B; V]$-typed subexpressions are finite-cut, then the given expression can be *fully discretized*.

Example 5.1. Consider a uniform distribution whose outcomes are compared to constants:

```
let x = uniform(0, 1) : float[B; T] in
  (x : float[B; T] < (0.3 : float[B; {0.3}])) ||
   (x : float[B; T] >= (0.5 : float[B; {0.5}]))
```

where $B = \{<0.3, <0.5\}$. The cut set B informs us that the probability of the comparisons evaluating to true is determined by the probability of x evaluating to one of the intervals

$$\mathrm{Intervals}(B)_0 = (-\infty, 0.3) \qquad \mathrm{Intervals}(B)_1 = [0.3, 0.5) \qquad \mathrm{Intervals}(B)_2 = [0.5, \infty)$$

induced by B. Hence, all expressions of type `float[B; ·]` can be discretized to expressions of the finite type $\mathbf{fin}(3)$, where $k_{\#3}$ represents the k-th interval $\mathrm{Intervals}(B)_k$ induced by B for $k \in \{0, 1, 2\}$.

Now consider the discretized expression obtained from computing $\mathcal{D}[\![e : \mathbf{bool}]\!]$, given by

```
let x = discrete(0.3, 0.2, 0.5) in (x <#3 1#3) || (x >=#3 2#3).
```

The probabilities appearing in $\mathbf{discrete}(\ldots)$ are determined by the cumulative distribution function (CDF) of the uniform distribution over $[0, 1]$, ensuring that the probability of x evaluating to $k_{\#3}$ in $\mathcal{D}[\![e : \mathbf{bool}]\!]$ *coincides* with the probability of x evaluating to a value in $\mathrm{Intervals}(B)_k$ in e. Hence, the constants 0.3 and 0.5 in e can be replaced by the constants $1_{\#3}$ and $2_{\#3}$ in $\mathcal{D}[\![e : \mathbf{bool}]\!]$.

Crucially, the *answers* constraint in the premise of the typing rules for comparisons (cf. Figure 6) ensures that B is a sufficiently fine-grained partition of $\mathbb{R}$, i.e., there is an injective function $\mathrm{ToInts} : \{0.3, 0.5\} \to \mathrm{Intervals}(B)$ that maps each constant $c \in \{0.3, 0.5\}$ to the unique interval in $\mathrm{Intervals}(B)$ containing c. This interval is determined by the cut convention: a cut $<c$ puts c in the interval to the right of the boundary, while a cut $\leq c$ puts c in the interval to the left.

Example 5.2. If the arguments to a uniform sampling are non-constant, the situation becomes more challenging. Consider the following typed expression $e : \mathbf{bool}$ (again only showing relevant types):

```
let x = (
        if uniform(0, 1) : float[{<0.5}; T] < (0.5 : float[{<0.5}; {0.5}]) then
        10 : float[{<=10,<=20}; {10}]
        else
        20 : float[{<=10,<=20}; {20}] ) : float[{<=10,<=20}; {10,20}] in

uniform(0 : float[{<=0}; {0}], x : float[{<=10,<=20}; {10,20}]) : float[{<5}; T]
```

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \mathbf{fin}(|B|+1) & \text{if } \tau = \mathbf{float}[B;V] \text{ and } B \neq \top \\ \mathbf{float}[B;V] & \text{if } \tau = \mathbf{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.*

6 Operational Semantics and Soundness

The soundness statement in this section concerns the *uniform fragment* of Slice: the syntax in Figure 5 without sequencing, non-strict comparisons, finite comparisons in source programs, or continuous samplers other than **`uniform`**. We write $Expr^{\mathsf{U}}_{\tau,\mathrm{cl}}$ for its closed, well-typed, fully annotated expressions of type τ. This fragment still includes discrete choice, higher-order functions, unbounded recursion, lists, and conditioning. We write $Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$ for the smallest closed, well-typed operational carrier that additionally contains the finite comparisons and core-language dispatch expressions generated by Figure 13; for this source fragment, those comparisons are finite strict comparison and finite equality, and that is closed under the small-step rules below. This carrier definition assumes preservation of closedness and typing by capture-avoiding substitution, by the transformation, and by every small-step rule. Those metatheoretic properties are required premises, not consequences of the grammar alone. The small-step tables are exhaustive for $Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$; constructs in the full implementation outside this fragment are not covered by the theorem below. Write $Vals^{\mathsf{U}}_{\tau,\mathrm{cl}}$ and $Vals^{\mathsf{U}+}_{\tau,\mathrm{cl}}$ for the values in $Expr^{\mathsf{U}}_{\tau,\mathrm{cl}}$ and $Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$, respectively. The intended correctness statement is that, for every $e : \mathbf{bool}$ in $Expr^{\mathsf{U}}_{\mathbf{bool},\mathrm{cl}}$,

$e : \mathbf{bool}$ *and* $\mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}$ *(obtained from Figure 13) are semantically equivalent.*

The first step is to make formal sense of semantic equivalence. For that, we define a monadic operational semantics in §6.1. Discretization decisions are driven by non-local, type-propagated information, so a local rule-by-rule argument is insufficient. A complete proof must relate the executions of e and $\mathcal{D}[\![e]\!]$ through the weak compiler simulation stated below. That simulation remains an open proof obligation.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle : Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} \to \mathcal{D}(Vals^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ in the operational closure to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. The probability of $\bot$ is the probability of an observation failure. Missing mass represents either non-termination or a runtime parameter error. The implementation distinguishes these cases operationally (by raising a runtime error for invalid sampling parameters), whereas this denotational model intentionally identifies them. With this model, semantic equivalence captures the sought-after correctness claim: if $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!] : Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} \to \mathcal{D}(Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

For each fixed full result type τ, the appendix uses canonical all-real-hole typed skeletons. Every occurrence of an exact-real atom gets a fresh coordinate: not only term-level float literals, but also discrete weights, cut boundaries, finite value-set members, and real atoms in all typing annotations, including the conclusion type. Required equality between coordinates and reconstruction of exactly τ are imposed in the filling domain. The remaining skeleton tags are therefore countable finite syntax, and the occurrence-indexed policy gives a unique tag and filling. For each skeleton s, its admissible filling set $A_{s;\tau} \subseteq \mathbb{R}^{m_s}$ records the actual typing, annotation, and transformation-side conditions. Conditional on the explicit obligation that every such set is Borel, the countable disjoint union of the fixed-skeleton spaces is a standard Borel carrier for $Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$. The paper does not infer Borelness merely from containment in Euclidean space.

Equipped with this family of measurable spaces, we define the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). For instance, if e_1 is not a value, then[1]

$$[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g. \begin{cases} \delta_\bot & \text{if } g = \bot \\ \delta_{(\mathbf{let}\ x = g\ \mathbf{in}\ e_2:\tau_2):\tau_2} & \text{otherwise}. \end{cases}}_{\text{standard probabilistic monadic}}$$

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding **let**-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by **observe** (false : **bool**) : **unit**. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\mathbf{observe}\ (\text{false} : \mathbf{bool}) : \mathbf{unit}]\!](\{\bot\}) = 1$, the same holds for the surrounding **let**-statement, i.e., also $[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. The Boolean operators use left-to-right short-circuit evaluation: false && e reduces to false and true || e reduces to true without evaluating e. The appendix constructs the measurable expression spaces and records the remaining small-step measurability obligation needed to make these binds well-defined.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. Let the ordinary kernel S_τ take one step at non-values and return δ_v at a value v. The strict bind from Definition F.17 extends S_τ internally at $\bot$. We lift it to an n-step semantics $\langle\cdot\rangle_n$ by:

$$\langle e : \tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e : \tau\rangle_{n+1} \;=\; \langle e : \tau\rangle_n \gg\!= S_\tau$$

Conditional on the small-step functional being a sub-Markov kernel (Theorem F.29), this iteration is well-defined. We then define for every type τ the functional $\langle\cdot\rangle : Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} \to \mathcal{D}(Vals^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot)$ as follows. Let $T_\tau = Vals^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot$ and restrict each n-step measure to T_τ. Values and $\bot$ are absorbing, so these terminal restrictions form an increasing sequence of subprobability measures. We define $\langle e : \tau\rangle(A) = \sup_{n\in\mathbb{N}} \langle e : \tau\rangle_n(A)$ for measurable $A \subseteq T_\tau$. The supremum is a subprobability measure. This argument uses monotonicity only after restriction to terminal outcomes; the full n-step measures on expressions need not be monotone on arbitrary measurable sets.

[1] Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A) \cdot \mu(dx)$, see Definition F.17 for details. δ_e denotes the Dirac distribution of e.

6.2 The remaining compiler-simulation obligation

The ordinary rules above define the behavior of both source and generated expressions. They do not by themselves prove that discretization preserves behavior. In particular, the generated term for a distribution with dynamic parameters takes deterministic administrative steps—let substitution, finite equality, Boolean conjunction, and conditional selection—for which the source has no matching step. A proof-only $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ operation that performs the complete dispatch in one step does not describe the term emitted by the transformation.

The appendix proves an abstract transfer theorem for any coherent measurable joint kernel. Applying it here still requires a measurable normalizer for the generated administrative reductions, a measurable related-state carrier, one measurable joint sub-Markov kernel with the source and collapsed-target kernels as exact marginals, terminal synchrony, equality of related terminal Boolean values, compatibility with the distinguished observation-failure outcome, and terminal cofinality between collapsed and ordinary target iteration. Equal failure and missing mass then follow from the transfer theorem; they are not premises smuggled into the simulation.

Conjecture 6.1 (Weak-simulation exactness). *The literal Slice transformation satisfies the measurable normalization, coherent-joint-kernel, terminal-compatibility, and cofinality premises above. Consequently, for every* $e : \mathtt{bool} \in Expr^{\cup}_{\mathtt{bool},\mathrm{cl}}$,

$$\langle e : \mathtt{bool} \rangle = \langle \mathcal{D}[\![e : \mathtt{bool}]\!] : \mathtt{bool} \rangle.$$

The earlier synchronous graph $\{(e, \mathcal{D}[\![e]\!])\}$ does not establish the hypothesis: it requires one source step and one target step at the same time, and it is not closed under the administrative reductions emitted for dynamic distribution parameters. Establishing the weak simulation remains an open proof obligation; therefore this paper does not presently claim unconditional exactness for the transformation.

7 Implementation, Case Studies, and Evaluation

We have implemented the Slice type inference and discretization procedures in OCaml. The implementation is open-source and available at https://github.com/[anonymized], together with all benchmarks, scripts, and a Docker image for reproduction. To compute the interval masses of continuous distributions, we use the OCaml GSL package [19] for cumulative distribution function calculations. All experiments were run in this Docker image on an Apple M3 with 16 GB of RAM.

This section is driven by two research questions:

(1) Does Slice enable exact inference on programs combining continuous sampling, higher-order functions, and unbounded recursion?
(2) On programs that are within the scope of existing tools for exact inference, is the combination of Slice and discrete inference back-ends competitive?

For the first question, we provide a collection of case studies that involve continuous distributions together with recursion, higher-order functions, lists, or conditioning in §7.1. For these case studies, we use Slice to transform the original programs to finite discrete models and analyze the resulting program with Storm [16] — a probabilistic model checker.

For the second question, we evaluate Slice+Dice and Slice+Roulette on two classes of benchmarks in Sections 7.2 and 7.3: (i) synthetic scaling benchmarks that let us vary program size and dependency structure in a controlled way, and (ii) benchmark suites from prior work [22, 40]. We compare against Sppl [40], an exact inference engine supporting continuous probabilistic programs, on benchmarks that *both* Slice and Sppl support, and we additionally report Psi and Hakaru timings

$\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \mathbf{float}[B_i; V_i]]\!] : \mathcal{D}[\![\mathbf{float}[B_i; V_i]]\!]$ for $i = 1, 2$. We consider each sub-case of the discretization function:

- $e_1, e_2 \in \mathbb{R}$ and $e_1 > e_2$: then $\mathcal{D}[\![\mathbf{uniform}(\ldots) : \mathbf{float}[B; \top]]\!] = \mathbf{diverge}$, which is well-typed at any type by Diverge.
- $B = \top$: then $\mathcal{D}[\![\mathbf{float}[B; \top]]\!] = \mathbf{float}$, and

$$\mathcal{D}[\![\mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]]\!] = \mathbf{uniform}\big(\mathcal{D}[\![e_1 : \mathbf{float}[B_1; V_1]]\!],\ \mathcal{D}[\![e_2 : \mathbf{float}[B_2; V_2]]\!]\big).$$

  Since recovers(B_1, V_1) and recovers(B_2, V_2) are preserved under discretization, the result is well-typed at $\mathbf{float}$ by Continuous-2.
- $B \neq \top$ and $e_1, e_2 \in \mathbb{R}$: then $\mathcal{D}[\![\mathbf{float}[B; \top]]\!] = \mathbf{fin}(|B| + 1)$, and

$$\mathcal{D}[\![\mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]]\!] = \mathbf{discrete}(p_0, \ldots, p_{|B|})$$

  where $p_k = \mathrm{CDF}(b_{k+1}) - \mathrm{CDF}(b_k)$. Each $p_k \in [0, 1]$ and $\sum_k p_k = 1$, so this is well-typed at $\mathbf{fin}(|B| + 1)$ by Discrete.
- $B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, and e_1 or e_2 is a variable: the algorithm generates conditionals over all pairs (v_i, w_j), each branch dispatching to a constant parameter in $\mathbf{uniform}$. By the previous sub-case, each branch $\mathcal{D}[\![\mathbf{uniform}(v_i, w_j) : \mathbf{float}[B; \top]]\!]$ is well-typed at $\mathbf{fin}(|B| + 1) = \mathcal{D}[\![\mathbf{float}[B; \top]]\!]$. The conditional structure is well-typed by If (as in the $\mathbf{if}$ case above), so the full expression is well-typed at $\mathcal{D}[\![\mathbf{float}[B; \top]]\!]$.

Case e is any other language construct (functions, pairs, lists, *etc.*). By definition of $\mathcal{D}[\![\cdot]\!]$, the discretization recurses structurally over e and τ. That is, $\mathcal{D}[\![e : \tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each subexpression and subtype of $e : \tau$, and $\mathcal{D}[\![\tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each component type of τ. The result then follows immediately by applying the inductive hypothesis to each subterm, analogous to the above cases.

In every case, the typing derivation is preserved under $\mathcal{D}[\![\cdot]\!]$. Therefore,

$$\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]. \qquad \square$$

F Soundness Obligations

Throughout this appendix, Expr denotes the closed, well-typed operational carrier $\mathit{Expr}_{\mathrm{cl}}^{\mathsf{U}+}$ from §6; thus Expr_τ means $\mathit{Expr}_{\tau,\mathrm{cl}}^{\mathsf{U}+}$. Likewise, Vals_τ and Skel_τ denote, respectively, the values in this fragment and the skeletons arising from this fragment. An unqualified typing judgment $e : \tau$ in this appendix therefore ranges over Expr_τ. Source-only domains are written explicitly as $\mathit{Expr}_{\mathrm{cl}}^{\mathsf{U}}$. We assume that capture-avoiding substitution, the transformation, and the small-step rules preserve closedness and typing, and that all-real-hole reconstruction covers this carrier. Borelness of every admissible filling domain, typed substitution, and the small-step kernel remain explicit conjectures below.

F.1 Measure Theory Preliminaries

We first give a brief overview of basic measure-theoretic terms and lemmas, and refer the reader to standard textbooks for further background [43, 44].

Topological space. A topological space is a pair $(X, \mathcal{T})$ where $\mathcal{T}$ is a collection of subsets of X, called open sets, such that $\emptyset, X \in \mathcal{T}$, and $\mathcal{T}$ is closed under arbitrary unions and finite intersections.

Since $\mathcal{C}$ is a σ-algebra on X containing $\mathcal{G}$, it appears in this intersection. Therefore,

$$\sigma(\mathcal{G}) \subseteq \mathcal{C}.$$

□

Definition F.5 (Sub-Markov kernels). Given measurable spaces (X, Σ_X) and (Y, Σ_Y), a *sub-Markov kernel* with source X and target Y is a function $P : X \times \Sigma_Y \to [0,1]$ (equivalently, $X \to \Sigma_Y \to [0,1]$) satisfying two conditions:

(1) *Subprobability measure:* For each source state $x \in X$, the map $A \mapsto P(x, A)$ is a subprobability measure.
(2) *Measurable function:* For each event $A \in \Sigma_Y$ in the target space, the map $x \mapsto P(x, A)$ must be a measurable function.

Definition F.6 (Projection with error). Let $(X + \bot, \Sigma_{X+\bot})$ and $(Y + \bot, \Sigma_{Y+\bot})$ be measurable spaces, with (X, Σ_X) and (Y, Σ_Y) measurable spaces as well. Let $\mu \in \mathcal{D}((X \times Y) + \bot)$. We define the first and second projection measures as:

(1) $\pi_1(\mu)(A) := \mu(A \times Y)$ for every measurable $A \in \Sigma_X$, and $\pi_1(\mu)(\bot) := \mu(\bot)$
(2) $\pi_2(\mu)(B) := \mu(X \times B)$ for every measurable $B \in \Sigma_Y$, and $\pi_2(\mu)(\bot) := \mu(\bot)$

Then, $\pi_1(\mu)$ is a measure on $(X + \bot, \Sigma_{X+\bot})$ and $\pi_2(\mu)$ is a measure on $(Y + \bot, \Sigma_{Y+\bot})$.

F.2 Measure Space on Expressions

The carrier construction must factor out every exact-real atom, not only term-level float constants. Otherwise real-valued weights or subtype annotations remain in the skeleton tag and the proposed skeleton index need not be countable.

Definition F.7 (All-real-hole typed skeletons). Fix a full annotated result type τ. An all-real-hole typed skeleton is obtained from a canonical finite elaborated derivation concluding $\vdash e : \tau$ by replacing every occurrence of an exact-real atom by a fresh occurrence-indexed hole. This includes term-level real literals, probabilities in finite discrete choices, real cut boundaries, real members of finite value-set annotations, and real atoms in the context, result type, and intermediate typing annotations. Every required equality between occurrences is recorded as a constraint on their separate coordinates.

Write $Skel_\tau$ for the resulting finite skeletons for this fixed full type τ. Skeleton tags retain constructors, rule names, de Bruijn indices, natural-number arities, and hole indices, but no real numbers. If $s \in Skel_\tau$ has m_s holes, let

$$A_{s;\tau} \subseteq \mathbb{R}^{m_s}$$

be the fillings satisfying all actual typing, annotation, and transformation-side conditions, including reconstruction of exactly the fixed conclusion type τ, and write $s[\vec{r}]$ for reconstruction. The canonical derivation and fresh occurrence-indexed policy give every expression a unique skeleton tag and filling.

Lemma F.8 (Countability of all-real-hole skeletons). *For every fixed full annotated result type τ, $Skel_\tau$ is countable.*

Proof. Finite trees over the countable alphabet of constructors, typing-rule names, de Bruijn indices, natural-number arities, and hole indices form a countable set. No exact-real atom remains in a skeleton tag. □

Conjecture F.9 (Borel filling domains). *For every $s \in Skel_\tau$, the set $A_{s;\tau}$ is a Borel subset of $\mathbb{R}^{m_s}$.*

This is a separate measure-theoretic obligation. It requires enumerating the actual typing, annotation, and transformation-side conditions and proving that each induces a Borel constraint; it is not implied merely by $A_{s;\tau} \subseteq \mathbb{R}^{m_s}$.

Definition F.10 (Fixed-skeleton carrier). Give $A_{s;\tau}$ the subspace Borel σ-algebra $\mathcal{F}_{s;\tau}$. Define

$$Expr_{s;\tau} = \{s[\vec{r}] \mid \vec{r} \in A_{s;\tau}\}$$

and transport $\mathcal{F}_{s;\tau}$ along the reconstruction bijection to obtain $\mathcal{F}_{Expr_{s;\tau}}$.

Lemma F.11 (Fixed-skeleton isomorphism). *Reconstruction $A_{s;\tau} \to Expr_{s;\tau}$ and decomposition $Expr_{s;\tau} \to A_{s;\tau}$ are inverse measurable maps by the transported σ-algebra.*

Definition F.12 (Expression carrier). Define

$$Expr_\tau = \bigsqcup_{s \in Skel_\tau} Expr_{s;\tau}, \qquad \mathcal{F}_{Expr_\tau} = \{A \mid A \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}} \text{ for every } s \in Skel_\tau\}.$$

Lemma F.13 (Conditional standard-Borel expression carrier). *If Theorem F.9 holds for every $s \in Skel_\tau$, then $Expr_\tau$ is a standard Borel space.*

Proof. Each $A_{s;\tau}$ is then a standard Borel subspace of a finite-dimensional Euclidean space, hence so is its isomorphic fixed-skeleton carrier. By Lemma F.8, $Expr_\tau$ is a countable disjoint union of standard Borel spaces, which is standard Borel. □

Definition F.14 (Extension with error element). Let $(Expr_\tau, \mathcal{F}_{Expr_\tau})$ be the measurable space of well-typed operationally closed expressions of type τ. Define

$$Expr_\tau + \bot := Expr_\tau \cup \{\bot\},$$

equipped with the σ-algebra

$$\mathcal{F}_{Expr_\tau + \bot} := \{A,\ A \cup \{\bot\} \mid A \in \mathcal{F}_{Expr_\tau}\}.$$

Lemma F.15. *$(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau + \bot})$ is a measurable space.*

Proof. We verify that $\mathcal{F}_{Expr_\tau + \bot}$ is indeed a σ-algebra on $Expr_\tau + \bot$.

By construction, every element of $\mathcal{F}_{Expr_\tau + \bot}$ is of the form A or $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$.

(i) Contains $Expr_\tau + \bot$: Take $A = Expr_\tau \in \mathcal{F}_{Expr_\tau}$. Then,

$$A \cup \{\bot\} = Expr_\tau \cup \{\bot\} = Expr_\tau + \bot \in \mathcal{F}_{Expr_\tau + \bot}.$$

(ii) Closure under complements: Let $C \in \mathcal{F}_{Expr_\tau + \bot}$. There are two cases:

- If $C = A$: Then, $(Expr_\tau + \bot) \setminus A = (Expr_\tau \setminus A) \cup \{\bot\}$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$, so $(Expr_\tau \setminus A) \cup \{\bot\}$ is of the form $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$ and hence lies in $\mathcal{F}_{Expr_\tau + \bot}$.
- If $C = A \cup \{\bot\}$: Then, $(Expr_\tau + \bot) \setminus (A \cup \{\bot\}) = Expr_\tau \setminus A$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$, so $Expr_\tau \setminus A$ is of the form A for some $A \in \mathcal{F}_{Expr_\tau}$ and hence lies in $\mathcal{F}_{Expr_\tau + \bot}$.

In both cases the complement lies in $\mathcal{F}_{Expr_\tau + \bot}$.

(iii) Closure under countable unions: Let $(C_n)_{n\in\mathbb{N}}$ be a sequence with $C_n \in \mathcal{F}_{Expr_\tau+\bot}$ for all n. For each n, define $A_n := C_n \cap Expr_\tau \in \mathcal{F}_{Expr_\tau}$, where the latter holds since C_n is of the form A or $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$, and in either case $C_n \cap Expr_\tau = A$.

Let $A := \bigcup_{n\in\mathbb{N}} A_n$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $A \in \mathcal{F}_{Expr_\tau}$. Now observe:

$$\bigcup_{n\in\mathbb{N}} C_n = A \cup \left(\bigcup_{n\in\mathbb{N}} (C_n \setminus Expr_\tau) \right).$$

The second factor is either $\emptyset$ (if every $C_n = A_n$) or $\{\bot\}$ (if some $C_n = A_n \cup \{\bot\}$). In the first sub-case, $\bigcup_n C_n = A$ is of the form A for some $A \in \mathcal{F}_{Expr_\tau}$; in the second, $\bigcup_n C_n = A \cup \{\bot\}$ is of the form $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$. In both cases $\bigcup_n C_n \in \mathcal{F}_{Expr_\tau+\bot}$. □

Finally, we can equip this measurable space $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ with a subprobability measure such that $\mu(Expr_\tau + \bot) \leq 1$.

We give some basic lemmas:

Lemma F.16. *Let $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ be the measurable space of well-typed operationally closed expressions with error from Definition F.14; it is standard Borel conditional on Theorem F.9. Let $Vals_\tau \subset Expr_\tau$ be the set of value expressions in this fragment. Let $N_\tau \subset Expr_\tau$, defined as $N = Expr_\tau \setminus Vals_\tau$, be the set of non-value expressions in this fragment. Let $\{\bot\}$ be the singleton set containing the error element $\bot$. The following are true:*

(1) The set $Vals_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.
(2) The set $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.
(3) The set $N_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.

Proof. We prove the measurability of $Vals_\tau$, $\{\bot\}$, and N_τ in separate parts. For each, we show that it belongs to the σ-algebra $\mathcal{F}_{Expr_\tau+\bot}$ on the extended expression space $Expr_\tau + \bot$.

(1) **(Measurability of $Vals_\tau$):** Every expression skeleton $s \in Skel_\tau$ is either a value skeleton or a non-value skeleton, depending on whether the syntactic form of s is a value. An expression $e \in Expr_\tau$ is a value iff its skeleton is a value skeleton.
To show $Vals_\tau \in \mathcal{F}_{Expr_\tau}$, by Definition F.12, it suffices to show that $\forall s \in Skel_\tau$, $Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
Fix $s \in Skel_\tau$:
- If s is a value skeleton, then every expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
- If s is a non-value skeleton, then no expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = \emptyset \in \mathcal{F}_{Expr_{s;\tau}}$.

Thus, $Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for any $s \in Skel_\tau$, so $Vals_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

(2) **(Measurability of $\{\bot\}$):** Taking $A = \emptyset$ in Definition F.14 gives $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.

(3) **(Measurability of N_τ):** $N_\tau = Expr_\tau \setminus Vals_\tau$ is the complement of $Vals_\tau$ in $Expr_\tau$. Since $Vals_\tau \in \mathcal{F}_{Expr_\tau}$ and $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $N_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

□

Note that in general, given (X, Σ_X) and (Y, Σ_Y) to be measurable spaces, $((X \times Y) + \bot)$ forms a measurable space, since the product $(X \times Y)$ is measurable and for any measurable space (Z, Σ_Z), the space $(Z + \bot, \Sigma_{Z+\bot})$ with $\Sigma_{Z+\bot}$ generated by $\Sigma_Z \cup \bot$ is measurable.

F.3 Distribution Monad with Error

Let X be a set equipped with a suitable measurable space so that $\mathcal{D}(X + \bot)$ denotes the set of subprobability measures over $X + \bot$. We use the following generic bind construction:

Definition F.17 (Distribution Monad with Error). We define

$$\begin{aligned}&\text{return}\colon X \to \mathcal{D}(X+\bot), \quad \text{return}(x) = \delta_x\\ &\text{bind}\colon \mathcal{D}(X+\bot) \to (X \to \mathcal{D}(Y+\bot)) \to \mathcal{D}(Y+\bot),\\ &\text{bind}(\mu)(f)(A) = \int_{X+\bot} f_\bot(x)(A)\cdot \mu(dx)\,,\end{aligned}$$

where δ_x denotes the Dirac measure for x, f is an ordinary sub-Markov kernel on X, and $f_\bot$ is its strict extension from Definition F.18. Thus bind propagates existing $\bot$ mass.

Definition F.18 ($\bot$-extension). Define the $\bot$-extension of $g : X \to \mathcal{D}(Y+\bot)$ to be $g_\bot\colon (X+\bot) \to \mathcal{D}(Y+\bot)$, obtained from g by:

$$g_\bot(e) = \begin{cases}\delta_\bot & \text{if } e = \bot\\ g(e) & \text{otherwise}\,.\end{cases}$$

Lemma F.19 ($\bot$-extension is a sub-Markov kernel). *Given a sub-Markov kernel $f : X \to \mathcal{D}(Y+\bot)$, its strict $\bot$ extension $f_\bot : (X+\bot) \to \mathcal{D}(Y+\bot)$ is also a sub-Markov kernel.*

Proof. We verify the two conditions for $f_\bot$ to be a sub-Markov kernel.

(1) Subprobability measure: We show for each $e \in X+\bot$, the map

$$A \mapsto f_\bot(e)(A)$$

is a subprobability measure.

Fix $e \in X+\bot$.

- If $e = \bot$, then $f_\bot(\bot) = \delta_\bot$, the Dirac measure at $\bot$, which is a subprobability measure on $Y+\bot$.
- If $e \in X$, then $f_\bot(e) = f(e)$, which is a subprobability measure by assumption.

Thus, for all $e \in X+\bot$, the map $A \mapsto f_\bot(e)(A)$ is a subprobability measure.

(2) Measurable function: We show for each $A \in \mathcal{F}_{Y+\bot}$, the map $e \mapsto f_\bot(e)(A)$ is a measurable function.

Fix $A \in \mathcal{F}_{Y+\bot}$. Define

$$g\colon (X+\bot) \to [0,1], \qquad g(e) := f_\bot(e)(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_{X+\bot}$ for every $B \in \mathcal{B}([0,1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}((a,\infty)\cap[0,1]) \in \mathcal{F}_{X+\bot}$$

because the family of sets $\{(a,\infty)\cap[0,1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0,1])$.

Because $g(e) \in [0,1]$ for all $e \in X+\bot$ by definition, we have for every $a \in \mathbb{R}$,

$$\begin{aligned}g^{-1}\big((a,\infty)\cap[0,1]\big) &= \{e \in X+\bot \mid g(e) \in (a,\infty)\cap[0,1]\}\\ &= \{e \in X+\bot \mid (g(e)\in(a,\infty)) \wedge (g(e)\in[0,1])\}\\ &= \{e \in X+\bot \mid (g(e) > a) \wedge (g(e)\in[0,1])\}\\ &= \{e \in X+\bot \mid g(e) > a\},\end{aligned}$$

where the last step uses that $g(e) \in [0,1]$ always holds. Hence it suffices to show that $\{e \in X+\bot \mid g(e) > a\} \in \mathcal{F}_{X+\bot}$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{e \in X+\bot \mid g(e) > a\} = \emptyset \in \mathcal{F}_{X+\bot}$.
- If $a < 0$: $\{e \in X+\bot \mid g(e) > a\} = X+\bot \in \mathcal{F}_{X+\bot}$.

- If $0 \le a < 1$: $\{e \in X + \bot \mid g(e) > a\}$.

The only nontrivial case is the last. It splits into two sets:

$$\begin{aligned}\{e \in X + \bot \mid g(e) > a\} &= \{e \in X + \bot \mid f_\bot(e)(A) > a\} \\ &= \{e \in X \mid f_\bot(e)(A) > a\} \cup \{\bot \mid f_\bot(e)(A) > a\} \\ &= \{e \in X \mid f(e)(A) > a\} \cup \{\bot \mid \delta_\bot(A) > a\}\end{aligned}$$

The first set is measurable in X (and hence $X + \bot$) since f is a sub-Markov kernel by assumption. The second set is either $\emptyset \in \mathcal{F}_{X+\bot}$ or $\{\bot\} \in \mathcal{F}_{X+\bot}$. Hence the required inverse image is measurable.

Since both conditions hold, $f_\bot$ is a sub-Markov kernel. □

Lemma F.20 (Strict-bind accounting). *For $\mu \in \mathcal{D}(X + \bot)$ and a sub-Markov kernel $f : X \to \mathcal{D}(Y + \bot)$, write*

$$\mathsf{fail}(\mu) = \mu(\{\bot\}), \qquad \mathsf{mass}(\mu) = \mu(X + \bot), \qquad \mathsf{miss}(\mu) = 1 - \mathsf{mass}(\mu).$$

Then the strict bind from Definition F.17 satisfies

$$\mathsf{fail}(\mu \gg\!= f) = \mathsf{fail}(\mu) + \int_X \mathsf{fail}(f(x))\, \mu(dx),$$

$$\mathsf{mass}(\mu \gg\!= f) = \mathsf{fail}(\mu) + \int_X \mathsf{mass}(f(x))\, \mu(dx),$$

$$\mathsf{miss}(\mu \gg\!= f) = \mathsf{miss}(\mu) + \int_X \mathsf{miss}(f(x))\, \mu(dx).$$

Proof. Split the defining integral over the measurable coproduct $X + \bot$. On X, the strict extension is f; at $\bot$ it is $\delta_\bot$. Evaluating first at $\{\bot\}$ and then at the whole space gives the first two identities. For the third, subtract the second identity from 1 and use $\mu(X + \bot) = \mu(X) + \mu(\{\bot\})$. □

Lemma F.21 (Dirac return is a sub-Markov kernel). *For any measurable space $(X, \mathcal{F}_X)$, the map*

$$\mathsf{return} : X \to \mathcal{D}(X + \bot), \qquad x \mapsto \delta_x.$$

is a sub-Markov kernel.

Proof. A Dirac probability measure is in particular a subprobability measure, and the measurability argument below establishes the second kernel condition.

(1) Probability measure: We show for each $x \in X$, the map

$$A \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a probability measure.

Fix $x \in X$. δ_x is the Dirac measure at x, hence a probability measure on $X + \bot$. Therefore, for every $x \in X$, the map $A \mapsto \delta_x(A)$ is a probability measure.

(2) Measurable function: We show that for each $A \in \mathcal{F}_{X+\bot}$, the map

$$x \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a measurable function.

Fix $A \in \mathcal{F}_{X+\bot}$. Define

$$g : X \to [0,1], \qquad g(x) := \delta_x(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0,1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}\big((a,\infty) \cap [0,1]\big) \in \mathcal{F}_X,$$

because the family $\{(a,\infty) \cap [0,1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0,1])$.

Because $g(e) \in [0,1]$ for all $e \in X + \bot$, we have for every $a \in \mathbb{R}$,

$$\begin{aligned} g^{-1}\big((a,\infty) \cap [0,1]\big) &= \{e \in X \mid g(e) \in (a,\infty) \cap [0,1]\} \\ &= \{e \in X \mid (g(e) \in (a,\infty)) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid (g(e) > a) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid g(e) > a\}, \end{aligned}$$

where the last step uses that $g(e) \in [0,1]$ always holds. Hence it suffices to show that $\{e \in X \mid g(e) > a\} \in \mathcal{F}_X$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{x \in X \mid g(x) > a\} = \emptyset \in \mathcal{F}_X$.
- If $a < 0$: $\{x \in X \mid g(x) > a\} = X \in \mathcal{F}_X$.
- If $0 \leq a < 1$: $\{x \in X \mid g(x) > a\}$.

The only nontrivial case is the last. Using $g(x) = \delta_x(A)$ and the fact that $\delta_x(A) \in \{0,1\}$ for all $x \in X$, we have for $0 \leq a < 1$:

$$\begin{aligned} \{x \in X \mid g(x) > a\} &= \{x \in X \mid \delta_x(A) > a\} \\ &= \{x \in X \mid \delta_x(A) = 1\} \\ &= \{x \in X \mid x \in A\} \\ &= A \cap X. \end{aligned}$$

Since $A \in \mathcal{F}_{X+\bot}$ and $\mathcal{F}_X = \{X \cap A \mid A \in \mathcal{F}_{X+\bot}\}$ is the subspace σ-algebra on $X \subseteq X + \bot$, it follows that $A \cap X \in \mathcal{F}_X$.

Therefore $\{x \in X \mid g(x) > a\} \in \mathcal{F}_X$ for all $a \in \mathbb{R}$, so $g^{-1}\big((a,\infty) \cap [0,1]\big) \in \mathcal{F}_X$ for all a. Hence $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0,1])$, and g is measurable. □

Lemma F.22 (Zero kernel is a sub-Markov kernel). *Let $(X, \mathcal{F}_X)$ and $(Y, \mathcal{F}_Y)$ be measurable spaces. The constant-zero map*

$$f : X \to \mathcal{D}(Y), \qquad f(x)(A) := 0$$

is a sub-Markov kernel.

Proof. We verify the two kernel conditions.

(1) Subprobability measure. We show for each $x \in X$, the map

$$A \mapsto f(x)(A)$$

is a subprobability measure.

Fix any $x \in X$. Then $f(x)$ is the zero measure: it satisfies σ-additivity since

$$f(x)\left(\bigcup_i A_i\right) = 0 = \sum_i 0 = \sum_i f(x)(A_i)$$

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

(2) Measurable function. For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on X, hence measurable.

Thus both conditions hold, so f is a sub-Markov kernel. □

Lemma F.23 (Composition of sub-Markov kernels). *Let (X, Σ_X), (Y, Σ_Y), and (Z, Σ_Z) be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \text{and} \quad Q : Y \to \mathcal{D}(Z)$$

are sub-Markov kernels, then their composition

$$(P \gg\!= Q)(x, A) := \int_Y Q(y, A)\, P(x, dy)$$

defines a sub-Markov kernel

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. See [30, Lemma 1.38]. □

F.4 Small-Step Semantics

Throughout this section, $e : \tau$ denotes a well-typed expression whose subexpressions are fully type-annotated. The dynamic-parameter transformation in Figure 13 denotes the displayed nest of core-language let-bindings , conditionals, Boolean conjunctions, finite equalities, and a discrete choice. No auxiliary one-step dispatch constructor is added to the language.

Equipped with these notions, we define the small-step semantics of the operationally closed fragment:

Definition F.24. For $e : \tau \in Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$, the small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in Tables 1 and 2.

The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., **uniform**$(0.8, 0.1)$). $[\![e : \tau]\!](\bot)$ is the probability of encountering an observation failure.

Lemma F.25 (Measurability of constant extension map). *Let $A \subseteq \mathbb{R}^n$ be equipped with the subspace σ-algebra*

$$\mathcal{F}_A := \{A \cap M \mid M \in \mathcal{B}(\mathbb{R}^n)\},$$

and let $B \subseteq \mathbb{R}^d$ be equipped with the subspace σ-algebra

$$\mathcal{F}_B := \{B \cap N \mid N \in \mathcal{B}(\mathbb{R}^d)\}.$$

Fix constant vectors $c_1 \in \mathbb{R}^{m_1}, \ldots, c_k \in \mathbb{R}^{m_k}$. Define

$$g : \mathbb{R}^n \to \mathbb{R}^d$$

to be any map obtained by concatenating blocks consisting of

- *copies of the input vector $\vec{r} \in \mathbb{R}^n$, and*
- *the constant vectors $c_1, \ldots, c_k$,*

in some fixed order. That is, $g(\vec{r})$ is formed by inserting the vectors $\vec{r}$ and c_j into a fixed sequence of blocks.

Assume $g(A) \subseteq B$ and define $\phi := g|_A : A \to B$. Then ϕ is measurable.

Example F.26 (Constant extension maps). The constant extension map allows constants to be inserted and the input vector $\vec{r} \in \mathbb{R}^n$ to appear multiple times. The following are examples of such maps:

$\mathbf{e} : \boldsymbol{\tau}$	**constraints**	$[\![\boldsymbol{e} : \boldsymbol{\tau}]\!]$
$\mathbf{diverge} : \tau$		$\lambda A.0$
$v : \tau$		$\delta_{v:\tau}$
$\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2 : \tau_2$	$e_1 = v$ e_1 no val.	$\delta_{e_2[v/x]:\tau_2}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{\mathbf{let}\ x=g\ \mathbf{in}\ e_2:\tau_2:\tau_2}$
$\mathbf{if}\ e_1 : \mathbf{bool}\ \mathbf{then}\ e_2 : \tau\ \mathbf{else}\ e_3 : \tau : \tau$	$e_1 = \text{true}$ $e_1 = \text{false}$ o.w.	$\delta_{e_2:\tau}$ $\delta_{e_3:\tau}$ $[\![e_1 : \mathbf{bool}]\!] \gg\!= \lambda g.\delta_{\mathbf{if}\ g\ \mathbf{then}\ e_2:\tau\ \mathbf{else}\ e_3:\tau:\tau}$
$\mathbf{observe}\ (e : \mathbf{bool}) : \mathbf{unit}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{():\mathbf{unit}}$ $\delta_{\bot}$ $[\![e : \mathbf{bool}]\!] \gg\!= \lambda g.\delta_{\mathbf{observe}\ (g:\mathbf{bool}):\mathbf{unit}}$
$e_1 : \tau_1 < e_2 : \tau_2 : \mathbf{bool}$ for $\tau_1 = \mathbf{float}[B; V_1], \tau_2 = \mathbf{float}[B; V_2]$	$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $e_1 \in Vals_{\tau_1}, e_2 \notin Vals_{\tau_2}$ $e_1 \notin Vals_{\tau_1}$	$\delta_{\text{true}:\mathbf{bool}}$ $\delta_{\text{false}:\mathbf{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{e_1:\tau_1<g:\mathbf{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g<e_2:\tau_2:\mathbf{bool}}$
$e_1 : \mathbf{fin}(n) <_{\#n} e_2 : \mathbf{fin}(n) : \mathbf{bool}$	$e_1 = i_{\#n}, e_2 = j_{\#n}, i < j$ $e_1 = i_{\#n}, e_2 = j_{\#n}, i \geq j$ $e_1 \in Vals_{\mathbf{fin}(n)}, e_2 \notin Vals_{\mathbf{fin}(n)}$ $e_1 \notin Vals_{\mathbf{fin}(n)}$	$\delta_{\text{true}:\mathbf{bool}}$ $\delta_{\text{false}:\mathbf{bool}}$ $[\![e_2 : \mathbf{fin}(n)]\!] \gg\!= \lambda g.\delta_{e_1<_{\#n}g:\mathbf{bool}}$ $[\![e_1 : \mathbf{fin}(n)]\!] \gg\!= \lambda g.\delta_{g<_{\#n}e_2:\mathbf{bool}}$
$e_1 : \mathbf{fin}(n) ==_{\#n} e_2 : \mathbf{fin}(n) : \mathbf{bool}$	$e_1 = i_{\#n}, e_2 = j_{\#n}, i = j$ $e_1 = i_{\#n}, e_2 = j_{\#n}, i \neq j$ $e_1 \in Vals_{\mathbf{fin}(n)}, e_2 \notin Vals_{\mathbf{fin}(n)}$ $e_1 \notin Vals_{\mathbf{fin}(n)}$	$\delta_{\text{true}:\mathbf{bool}}$ $\delta_{\text{false}:\mathbf{bool}}$ $[\![e_2 : \mathbf{fin}(n)]\!] \gg\!= \lambda g.\delta_{e_1==_{\#n}g:\mathbf{bool}}$ $[\![e_1 : \mathbf{fin}(n)]\!] \gg\!= \lambda g.\delta_{g==_{\#n}e_2:\mathbf{bool}}$
$\mathbf{uniform}(e_1 : \tau_1, e_2 : \tau_2) : \mathbf{float}[B; V]$	$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $e_1 \in Vals_{\tau_1}, e_2 \notin Vals_{\tau_2}$ $e_1 \notin Vals_{\tau_1}$	$\text{Uniform}(v_1, v_2) \gg\!= \lambda v.\delta_{(v:\mathbf{float}[B;V])}$ $\lambda A.0$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{\mathbf{uniform}(e_1:\tau_1,g):\mathbf{float}[B;V]}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{\mathbf{uniform}(g,e_2:\tau_2):\mathbf{float}[B;V]}$
$\mathbf{discrete}(v_0, \ldots, v_{n-1}) : \mathbf{fin}(n)$	$v_i \geq 0, \sum_i v_i = 1$ o.w.	$\text{Discrete}(v_0, \ldots, v_{n-1})$ $\lambda A.0$

Table 1. Small-step operational semantics. In the last case, the existence of suitable v_i and w_j is guaranteed since recovers(B_i, V_i) and $e_i \in V_i$ for $i \in \{1, 2\}$. Here, every function of the form $f(g) = \lambda g.\delta_{\ldots}$ is extended to $f_\bot$ in the sense of Definition F.18.

- $g : \mathbb{R}^n \to \mathbb{R}^n, \qquad g(\vec{r}) = \vec{r}$
- $g : \mathbb{R}^n \to \mathbb{R}^{n+m_1}, \qquad g(\vec{r}) = (\vec{r}, c_1), \quad c_1 \in \mathbb{R}^{m_1}$
- $g : \mathbb{R}^n \to \mathbb{R}^{m_1+n+m_2}, \qquad g(\vec{r}) = (c_1, \vec{r}, c_2), \quad c_1 \in \mathbb{R}^{m_1},\ c_2 \in \mathbb{R}^{m_2}$
- $g : \mathbb{R}^n \to \mathbb{R}^{n+m_2}, \qquad g(\vec{r}) = (\vec{r}, c_2), \quad c_2 \in \mathbb{R}^{m_2}$
- $g : \mathbb{R}^n \to \mathbb{R}^{m_1+n+m_2+n+m_3+n}, \qquad g(\vec{r}) = (c_1, \vec{r}, c_2, \vec{r}, c_3, \vec{r}), \quad c_i \in \mathbb{R}^{m_i}$

Proof. We show that $\phi^{-1}(S) \in \mathcal{F}_A$ for each $S \in \mathcal{F}_B$.

$\mathbf{e} : \boldsymbol{\tau}$	**constraints**	$[\![e : \tau]\!]$
$e_1 : \tau_1$ && $e_2 : \tau_2 : \mathbf{bool}$ $\tau_1 = \mathbf{bool}, \tau_2 = \mathbf{bool}$	$e_1 = \text{false}$ $e_1 = \text{true}$ o.w.	$\delta_{\text{false}:\mathbf{bool}}$ $\delta_{e_2:\mathbf{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g \&\& e_2:\tau_2:\mathbf{bool}}$
$e_1 : \tau_1$ || $e_2 : \tau_2 : \mathbf{bool}$ $\tau_1 = \mathbf{bool}, \tau_2 = \mathbf{bool}$	$e_1 = \text{true}$ $e_1 = \text{false}$ o.w.	$\delta_{\text{true}:\mathbf{bool}}$ $\delta_{e_2:\mathbf{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g \mid\mid e_2:\tau_2:\mathbf{bool}}$
not $e : \tau : \mathbf{bool}$ $\tau = \mathbf{bool}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{\text{false}:\mathbf{bool}}$ $\delta_{\text{true}:\mathbf{bool}}$ $[\![e : \tau]\!] \gg\!= \lambda g.\delta_{\text{not } g:\mathbf{bool}}$
$(e_1 : \tau_1, e_2 : \tau_2) : \tau_1 * \tau_2$	$e_1 = v_1, e_2 = v_2$ $e_1 \in \mathit{Vals}_{\tau_1}, e_2 \notin \mathit{Vals}_{\tau_2}$ $e_1 \notin \mathit{Vals}_{\tau_1}$	$\delta_{(v_1,v_2):\tau_1*\tau_2}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{(e_1:\tau_1, g:\tau_2):\tau_1*\tau_2}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{(g:\tau_1, e_2:\tau_2):\tau_1*\tau_2}$
$\mathbf{fst}\ (e : \tau_1 * \tau_2) : \tau_1$	$e = (v_1, v_2)$ o.w.	$\delta_{v_1:\tau_1}$ $[\![e : \tau_1 * \tau_2]\!] \gg\!= \lambda g.\delta_{\mathbf{fst}\ g:\tau_1}$
$\mathbf{snd}\ (e : \tau_1 * \tau_2) : \tau_2$	$e = (v_1, v_2)$ o.w.	$\delta_{v_2:\tau_2}$ $[\![e : \tau_1 * \tau_2]\!] \gg\!= \lambda g.\delta_{\mathbf{snd}\ g:\tau_2}$
$(e_1 : \tau_1 \to \tau_2\ e_2 : \tau_1) : \tau_2$	$e_1 = \mathbf{fun}\ x \to e, e_2 = v_2$ $e_1 = \mathbf{fix}\ f\ x := e, e_2 = v_2$ $e_1 \in \mathit{Vals}_{\tau_1 \to \tau_2}, e_2 \notin \mathit{Vals}_{\tau_1}$ $e_1 \notin \mathit{Vals}_{\tau_1 \to \tau_2}$	$\delta_{e[v_2/x]:\tau_2}$ $\delta_{e[v_2/x,(\mathbf{fix}\ f\ x:=e)/f]:\tau_2}$ $[\![e_2 : \tau_1]\!] \gg\!= \lambda g.\delta_{(e_1:\tau_1\to\tau_2)\ (g:\tau_1):\tau_2}$ $[\![e_1 : \tau_1 \to \tau_2]\!] \gg\!= \lambda g.\delta_{(g:\tau_1\to\tau_2)\ (e_2:\tau_1):\tau_2}$
nil : $\mathbf{list}(\tau)$		$\delta_{\text{nil}:\mathbf{list}(\tau)}$
$(e_1 : \tau :: e_2 : \mathbf{list}(\tau)) : \mathbf{list}(\tau)$	$e_1 = v_1, e_2 = v_2$ $e_1 \in \mathit{Vals}_\tau, e_2 \notin \mathit{Vals}_{\mathbf{list}(\tau)}$ $e_1 \notin \mathit{Vals}_\tau$	$\delta_{v_1::v_2:\mathbf{list}(\tau)}$ $[\![e_2 : \mathbf{list}(\tau)]\!] \gg\!= \lambda g.\delta_{e_1:\tau:g:\mathbf{list}(\tau)}$ $[\![e_1 : \tau]\!] \gg\!= \lambda g.\delta_{(g:\tau)::(e_2:\mathbf{list}(\tau)):\mathbf{list}(\tau)}$
$(\mathbf{match}\ e : \mathbf{list}(\tau_1)\ \mathbf{with}$ $\quad \mid \text{nil} \to e_1 : \tau_2$ $\quad \mid h :: t \to e_2 : \tau_2$ $\quad \mathbf{end}) : \tau_2$	$e = \text{nil}$ $e = v_h :: v_t$ o.w.	$\delta_{e_1:\tau_2}$ $\delta_{e_2[v_h/h, v_t/t]:\tau_2}$ $[\![e : \mathbf{list}(\tau_1)]\!] \gg\!= \lambda g.$ $\delta_{(\mathbf{match}\ g\ \mathbf{with}\ \mid \text{nil} \to e_1\ \mid h::t \to e_2\ \mathbf{end}):\tau_2}$

Table 2. Small-step operational semantics (continued). Here, every function of the form $f(g) = \lambda g.\delta_{\ldots}$ is extended to $f_\perp$ in the sense of Definition F.18.

Fix $S \in \mathcal{F}_B$. By definition of $\mathcal{F}_B$, there exists $N \in \mathcal{B}(\mathbb{R}^d)$ such that $S = B \cap N$. We compute:

$$
\begin{aligned}
\phi^{-1}(S) &= \{\vec{r} \in A \mid \phi(\vec{r}) \in B \cap N\} \\
&= \{\vec{r} \in A \mid \phi(\vec{r}) \in B \cap \phi(\vec{r}) \in N\} && \text{(definition of } \cap) \\
&= \{\vec{r} \in A \mid \phi(\vec{r}) \in N\} && (\phi(\vec{r}) \in B \text{ for all } \vec{r} \in A) \\
&= \{\vec{r} \in A \mid g(\vec{r}) \in N\} && \text{(definition of } \phi) \\
&= \{\vec{r} \in A \mid g(\vec{r}) \in N\} \cap \mathbb{R}^n && (\vec{r} \text{ ranges over } A \subseteq \mathbb{R}^n) \\
&= A \cap \{\vec{r} \in \mathbb{R}^n \mid g(\vec{r}) \in N\}. && \text{(rewrite as } \cap \text{ with } A)
\end{aligned}
$$

It remains to show that $\{\vec{r} \in \mathbb{R}^n \mid (\vec{r}, \vec{c}) \in N\} \in \mathcal{B}(\mathbb{R}^n)$. Define the total constant-extension map

$$g \colon \mathbb{R}^n \to \mathbb{R}^d.$$

We claim g is continuous. A map into a product space is continuous if and only if each coordinate function is continuous [36, Theorem 19.6]. The coordinate functions of g are:

- the identity $\mathrm{id} \colon \mathbb{R}^n \to \mathbb{R}^n$, $\vec{r} \mapsto \vec{r}$, and
- the constant map $\kappa_{\vec{c}_i} \colon \mathbb{R}^n \to \mathbb{R}^{m_i}$, $\vec{r} \mapsto \vec{c}_i$.

Both are continuous: for any open $U \subseteq \mathbb{R}^n$, $\mathrm{id}^{-1}(U) = U$ is open; and for any open $V \subseteq \mathbb{R}^{m_i}$,

$$\kappa_{\vec{c}_i}^{-1}(V) = \begin{cases} \mathbb{R}^n & \text{if } \vec{c}_i \in V, \\ \emptyset & \text{if } \vec{c}_i \notin V, \end{cases}$$

both of which are open. Hence g is continuous, and therefore Borel measurable. Applying this to $N \in \mathcal{B}(\mathbb{R}^d)$:

$$\{\vec{r} \in \mathbb{R}^n \mid g(\vec{r}) \in N\} = g^{-1}(N) \in \mathcal{B}(\mathbb{R}^n).$$

Hence $\phi_{\vec{c}}^{-1}(S) = A \cap g^{-1}(N) \in \mathcal{F}_A$ by definition, so $\phi_{\vec{c}}$ is measurable. □

Conjecture F.27 (Substitution is measurable). *Fix a typed expression $e : \tau_2$ and a variable $x : \tau_1$. The substitution map*

$$\mathrm{Sub}_{e,x} \colon Val_{\tau_1} \to Expr_{\tau_2}, \qquad \mathrm{Sub}_{e,x}(v) := e[v/x],$$

is measurable, where $Val_{\tau_1} \subseteq Expr_{\tau_1}$ carries the subspace σ-algebra inherited from $(Expr_{\tau_1}, \mathcal{F}_{Expr_{\tau_1}})$.

Conjecture F.28 (Measurability of operational contexts). *For every occurrence of a monadic bind $\mu \gg\!= f$ in the small-step operational semantics in Tables 1 and 2, $f : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a sub-Markov kernel. This is the ordinary continuation on $Expr_{\tau_1}$; strict bind supplies its bottom extension $f_\bot$.*

The skeleton construction reduces this claim to measurability of the corresponding context maps on each skeleton component. The argument is straightforward for fixed syntactic contexts, but a complete case analysis is not supplied here.

Conjecture F.29 (Small-step measurability). *For each type τ, the small-step operational semantics $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ defines a sub-Markov kernel.*

This requires, in addition to Theorem F.28, measurability of the sampler kernels as their parameters vary and measurability of the case partition used by the small-step tables. These measure-theoretic obligations remain open.

F.5 Big-Step Semantics

Conditional on Theorem F.29, the complete ordinary small-step tables above can be iterated without assuming a synchronous coupling. Define the ordinary kernel $S_\tau : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ to take one small step at non-values and to return δ_v at values. Its strict extension $(S_\tau)_\bot$ makes values and $\bot$ absorbing. With the strict bind from Definition F.17, define

$$\mu_0^e = \delta_e \qquad \text{and} \qquad \mu_{n+1}^e = \mu_n^e \gg\!= S_\tau.$$

Let $T_\tau = Vals_\tau + \bot$ and define the terminal restriction $\nu_n^e(A) = \mu_n^e(A \cap T_\tau)$. Absorption implies $\nu_n^e(A) \le \nu_{n+1}^e(A)$ for every measurable $A \subseteq T_\tau$. Notice that this statement is only about terminal restrictions: the full measures μ_n^e need not be monotone on arbitrary measurable sets.

Definition F.30 (Big-step semantics). For $e : \tau \in Expr_\tau$, define

$$\langle e : \tau \rangle(A) = \sup_{n \in \mathbb{N}} \nu_n^e(A) \qquad (A \subseteq T_\tau \text{ measurable}).$$

Because $(\nu_n^e)_n$ is an increasing sequence of subprobability measures, its pointwise supremum is a subprobability measure. Missing mass represents execution that has not reached a value or observation failure in any finite number of steps, including divergence and runtime parameter errors in this model. This construction supplies the ordinary big-step semantics; it does not establish a relation between a source expression and its generated target.

Theorem F.31 (Coherent coupling transfer to terminal semantics). *Let $S : X \to \mathcal{D}(X)$ and $T : Y \to \mathcal{D}(Y)$ be measurable sub-Markov kernels. Let $F_X \subseteq X$ and $F_Y \subseteq Y$ be measurable pointwise-absorbing terminal sets: $S(x) = \delta_x$ for every $x \in F_X$ and $T(y) = \delta_y$ for every $y \in F_Y$. Let $R \subseteq X \times Y$ be a measurable relation that is terminal-synchronous: $(x, y) \in R$ implies $x \in F_X$ iff $y \in F_Y$. Suppose there is one measurable sub-Markov kernel*

$$C : R \longrightarrow \mathcal{D}(R)$$

whose marginals factor through the ordinary kernels, so for every $(x, y) \in R$ and measurable $A \subseteq X$, $B \subseteq Y$,

$$C((x, y), \pi_X^{-1}(A)) = S(x, A), \qquad C((x, y), \pi_Y^{-1}(B)) = T(y, B).$$

For every $(x_0, y_0) \in R$, the terminal big-step semantics of x_0 and y_0 are the marginals of a terminal subprobability coupling supported on $R \cap (F_X \times F_Y)$. In particular, their terminal total masses, and hence their aggregate missing masses, are equal. No almost-sure termination premise is required.

Proof. Put $\eta_0 = \delta_{(x_0, y_0)}$ and define ordinary kernel iteration by $\eta_{n+1}(A) = \int_R C(r, A)\, \eta_n(dr)$. Induction on n shows that the marginals of η_n are the n-step measures generated by S and T. For example,

$$\begin{aligned}(\pi_X)_* \eta_{n+1}(A) &= \int_R C(r, \pi_X^{-1}(A))\, \eta_n(dr) \\ &= \int_R S(\pi_X r, A)\, \eta_n(dr) = \int_X S(x, A)\, (\pi_X)_* \eta_n(dx),\end{aligned}$$

and similarly for Y. Terminal synchrony and pointwise absorption make the restrictions of η_n to $R \cap (F_X \times F_Y)$ increasing. Their pointwise supremum is therefore a subprobability measure. Taking marginals commutes with these increasing suprema and yields exactly the two terminal big-step measures. Since both are marginals of the same terminal coupling, they have equal total mass; subtracting from 1 gives equal missing mass. □

If measurable failure sets $B_X \subseteq F_X$ and $B_Y \subseteq F_Y$ also satisfy $x \in B_X$ iff $y \in B_Y$ for every $(x, y) \in R$, the same coupling gives equal explicit failure mass by evaluation on $R \cap (B_X \times B_Y)$. This conclusion concerns the represented failure outcome. Divergence and other killed execution remain aggregate missing mass and are not identified with failure.

Corollary F.32 (Collapsed-to-ordinary terminal transfer). *Suppose Theorem F.31 applies to a source kernel S and a measurable collapsed target kernel $\overline{T}$. If the ordinary target kernel T is*

terminal-cofinal with $\overline{T}$, meaning that for every initial target y and measurable terminal event $A \subseteq F_Y$,

$$\sup_n \overline{T}^{(n)}(y, A) = \sup_n T^{(n)}(y, A),$$

then the transferred target marginal is the ordinary target terminal semantics.

Proof. The theorem identifies the source terminal semantics with that of $\overline{T}$. The displayed cofinality premise identifies the latter measure, event by event, with the ordinary target terminal semantics. □

F.6 Weak Compiler-Simulation Obligation

The transformation shown in Figure 13 emits core-language let-bindings, conditionals, Boolean conjunctions, finite equalities, and discrete choices. It does not emit the auxiliary $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ constructor used by the earlier synchronous proof. Consequently, the graph $\{(e, \mathcal{D}[\![e]\!])\}$ is not closed under one ordinary step: target dispatch can take several deterministic steps while the source does not move.

Let $\longrightarrow_a$ contain the deterministic administrative target reductions for let substitution, finite equality, Boolean conjunction, and conditional selection. A noncircular application of Theorems F.31 and F.32 requires, for each type τ:

(1) Borelness of every all-real-hole filling domain $A_{s,\tau}$, as required by Theorem F.9, and the resulting standard-Borel expression carrier;
(2) a measurable administrative normalizer N_τ that terminates on generated administrative terms, and a measurable collapsed target kernel $\overline{S}_\tau$ obtained from the ordinary target semantics through that normalizer;
(3) a measurable related-state carrier R_τ containing each initial pair $(e, \mathcal{D}[\![e]\!])$ under consideration;
(4) one measurable joint sub-Markov kernel on R_τ whose marginals are exactly the source kernel and $\overline{S}_\tau$ and whose support remains in R_τ;
(5) terminal synchrony, equality of related terminal Boolean values, and compatibility of the distinguished observation-failure outcomes;
(6) terminal cofinality between $\overline{S}_\tau$ and the ordinary target kernel.

Under these premises, equal explicit observation-failure mass and equal aggregate missing mass are conclusions of the coherent transfer theorem, not simulation assumptions. The finite-step marginal equalities and their terminal-limit transfer are likewise conclusions of Theorem F.31, not additional conjecture premises.

Conjecture F.33 (Weak-simulation exactness). *The literal Slice transformation satisfies the measurable normalization, joint-kernel, terminal-synchrony, and cofinality premises above for every $e : \mathtt{bool} \in Expr^{\cup}_{\mathtt{bool},\mathrm{cl}}$, and consequently $\langle e : \mathtt{bool}\rangle = \langle \mathcal{D}[\![e : \mathtt{bool}]\!] : \mathtt{bool}\rangle$.*

The synchronous coupling tables and their projection argument are therefore not part of the current proof. They relied on a proof-only $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ step and did not account for stuttering in the literal generated target. The conjecture above records the required theorem; its formal statement and proof remain open.